

CO	Course Outcome
CO1	Define [L1: Remember] Algorithms, Asymptotic Notations, Recurrences, Advanced Data Structures, Algorithm design approaches and complexity classes.
CO2	Explain [L2: Understand] Various Algorithm design approaches and advanced data structures.
CO3	Apply [L3: Apply] appropriate algorithm design approach to solve a computational problem. Ability to apply a suitable recurrence solving method to solve a recurrence.
CO4	Analyse [L4: Analyze] an Algorithm to solve a computational problem.

Time: 1.5 Hrs.

M. M. 20

Section A

Q1. Attempt all questions:

(1X5 = 5 Marks)

- a) Define Head and Tail Recursion with example. CO1
- b) Arrange the following expressions by their asymptotic growth and justify your answer. CO2
 $2^n, \sqrt{\log n}, (2n)!, \log^2 n, 2^{\log^2 n}, n, 2^{2n}, 4^{\log n}$
- c) State Divide and Conquer approach with example. CO1
- d) For the given function $f(n)=2^{(n+1)}$ and $g(n)=2^n$, Show that $f(n) \in O(g(n))$. CO2
- e) Illustrate a R-B Tree with n internal nodes has height(h) of at most $2\log(n+1)$. CO3

Section B

Q2. Attempt all questions:

(2.5X4 = 10 Marks)

- ai) The recurrence $T(n) = 7T(n/3) + n^2$ describes the running time of an algorithm A. CO2
 Another competing algorithm B has a running time of $S(n) = aS(n/9) + n^2$. Identify the minimum value of 'a' such that A is asymptotically faster than B.

Or

- ii) Demonstrate the procedure of Heap Sort on the given array named as Arr and explain the running time complexity (Tighter Bound) of Build-Max-Heap(A, n) is $O(n)$. CO2
 Arr: <4, 1, 3, 2, 16, 9, 10, 14, 8, 7>.

- bi) Illustrate the Quick Sort algorithm and sort the given array named 'Arr' and compute its running time complexity. Arr: <5, 3, 2, 6, 9, 10, 11, 14> CO3

Or

- ii) Illustrate the time complexity of recurrence relation by using Master theorem. CO3

a) $T(n) = 9T\left(\frac{n}{3}\right) + n$

b) $T(n) = 5T\left(\frac{n}{2}\right) + n^2$

- ci) Solve the Following recurrence relation. CO3

$$T(n) = \begin{cases} 1 & \text{for } n = 1 \\ T\left(\frac{n}{2}\right) + T\left(\frac{n}{4}\right) + T\left(\frac{n}{8}\right) + n & \text{for } n > 1 \end{cases}$$

Or

- ii) Illustrate the Merge Sort algorithm on the given array named 'Arr' and compute its running time complexity. Arr: <41, 20, 30, 11, 10, 40, 31, 21>. CO3

- di) Illustrate the algorithm for Build_Min_Heap() on following keys and analyze its running time. CO3
 $\langle 25, 3, 16, 20, 2, 13, 23, 1 \rangle$

Or

- ii) Solve the following recurrence: CO3

$$T(n) = \begin{cases} 1 & \text{for } n = 1 \\ T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + n & \text{for } n > 1 \end{cases}$$

Section C

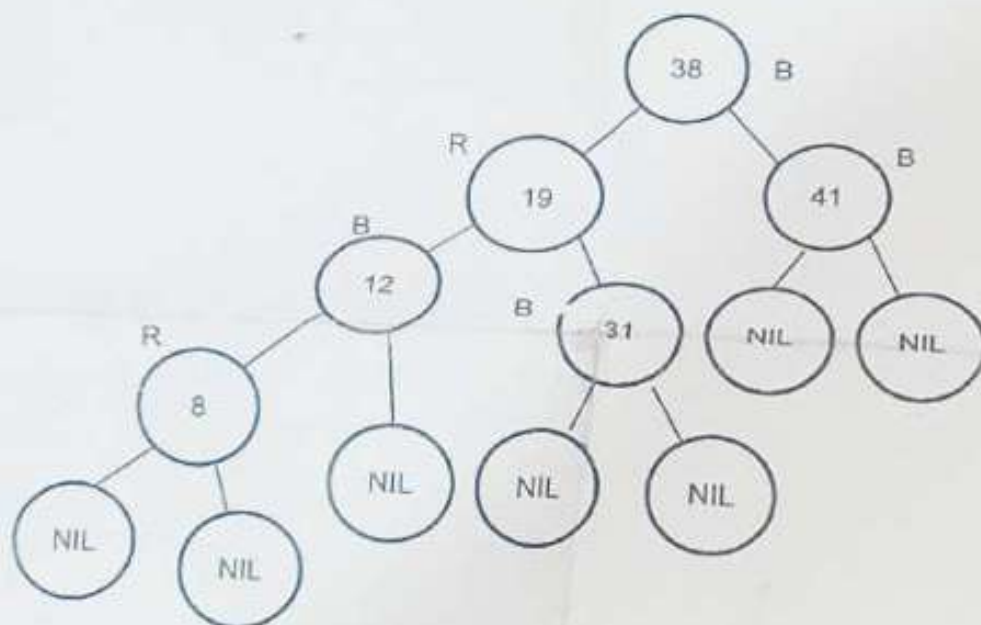
Q3. (5X1 = 5 Marks)

- i) Construct a B-Tree of Order 5 on the following data set and assume that B-Tree is initially empty. CO4

$\langle F, S, Q, K, C, L, H, T, V, W, M, R, N, P, A, B, X, Y, Z, D, E \rangle$

OR

- ii) Illustrate deletion algorithm of R-B Tree and delete the following keys 8, 12, 19, 31, 38, 41 from the following given R-B Tree. CO4



PRANVEER SINGH INSTITUTE OF TECHNOLOGY KANPUR

Odd Semester

Session 2024-25

CT -1

B. Tech 5th Semester**Design & Analysis of Algorithms (BCS-503)**

CO's	Course Outcomes
CO1	Define [L1: Remember] Algorithms, Asymptotic Notations, Recurrences, Advanced Data Structures, Algorithm design approaches and complexity classes.
CO2	Explain [L2: Understand] Various Algorithm design approaches and advanced data structures.
CO3	Apply [L3: Apply] appropriate algorithm design approach to solve a computational problem. Ability to apply a suitable recurrence solving method to solve a recurrence.
CO4	Analyse [L4: Analyze] an Algorithm to solve a computational problem.

Time: 1.5 Hrs.

M. M. 20

Section A**Q1. Attempt all questions:****(1X5 = 5 Marks)**

- a) If $f(n) = 2^n + n^2$ and $g(n) = 2^n$ show that $f(n) \in \Omega(g(n))$ CO2
- b) Arrange the following function in decreasing order CO2
 $< 2^n, n^3, n^2 \log n, n!, n \lg n, \lg \lg n, \frac{n}{2^n}, \sqrt{n} >$
- c) State the definition of following asymptotic notation: CO1
 i) O (Big Oh)
 ii) Ω (Omega)
- d) Compute the time complexity of the following function CO3
`Function_XYZ()`

```
{
  int i, j, k, n;
  for (i = n; i >= n/2; i = i/2)
  {
    for (j = 1; j <= i^2; j = j * 2)
    {
      for (k = 2; k <= n; k = k^2)
        printf("PSIT")
    }
  }
}
```
- e) Show that "There are at most $\lceil n/2^{h+1} \rceil$ nodes of height h in any n element heap". CO3

Section B**Q2. Attempt all questions:****(2.5 X4 = 10 Marks)**

- a i) Solve the following recurrence.
- CO3

$$T(n) = \begin{cases} 1 & \text{for } n = 1 \\ T\left(\frac{n}{2}\right) + T\left(\frac{n}{4}\right) + T\left(\frac{n}{8}\right) + n & \text{for } n > 1 \end{cases}$$

- Or**
- ii) Solve the following recurrence using recursion Tree method CO3
- $$T(n) = \begin{cases} 1 & \text{for } n = 1 \\ 3T\left(\frac{n}{4}\right) + cn^2 & \text{for } n > 1 \text{ and } c > 0 \end{cases}$$
- b i) Solve the following recurrence by using iterative method. CO3
- $$T(n) = \begin{cases} 1 & \text{for } n = 1 \\ 7T\left(\frac{n}{2}\right) + \Theta(n^2) & \text{for } n > 1 \text{ and } c > 0 \end{cases}$$
- (Note: $\log_2 7 = 2.81$)
- Or**
- ii) Solve the following recurrence by using iterative method CO3
- $$T(n) = \begin{cases} 1 & \text{for } n = 1 \\ T(n-1) + \log n & \text{for } n > 1 \end{cases}$$
- c i) Examine that $f(n) = 2n^2 + 3n + 4 \in O(n^2)$ CO4
- Or**
- ii) If $f(n) = 3n + 2$, $g(n) = n^2$ CO4
Examine that $f(n) \notin \Omega(g(n))$
- d i) Formulate the head iterative equation of insertion sort after illustrating the algorithm on the given list. Also, demonstrate its time complexity. CO3
- <9, 6, 5, 0, 8, 2, 7>
- Or**
- ii) Formulate the head iterative equation of insertion sort after illustrating the algorithm on the given list. Also, demonstrate its time complexity. CO3
- <17, 26, 13, 55, 34, 48>

Section C

Q3

(5X1 = 5 Marks)

- i) Consider the following array elements. CO3

< 82, 90, 10, 12, 15, 77, 55, 23, k>

(Note: k is the last two digit of your university roll number)

Describe and demonstrate the heapsort algorithm to ascertain the least number of interchanges done by Maxheapify() to transform the array into a Max-heap. Also show that the time complexity of heap sort in worst case is $O(n \log n)$.

Or

- ii) Describe a fast algorithm that improves the time complexity of insertion sort in average case. Demonstrate the same on the following array in ascending order. Provide the worst-case, best case and average case time complexity in general. CO3

< 35, 33, 42, 10, 14, 19, 27, 44>